SlimFL: Federated Learning With Superposition Coding Over Slimmable Neural Networks

Won Joon Yun, *Graduate Student Member, IEEE*, Yunseok Kwak, Hankyul Baek, *Graduate Student Member, IEEE*, Soyi Jung, *Member, IEEE*, Mingyue Ji, *Member, IEEE*, Mehdi Bennis, *Fellow, IEEE*, Jihong Park, *Senior Member, IEEE*, and Joongheon Kim, *Senior Member, IEEE*

***Abstract*— Federated learning (FL) is a key enabler for efficient communication and computing, leveraging devices' distributed computing capabilities. However, applying FL in practice is challenging due to the local devices' heterogeneous energy, wireless channel conditions, and non-independently and identically distributed (non-IID) data distributions. To cope with these issues, this paper proposes a novel learning framework by integrating FL and width-adjustable slimmable neural networks (SNN). Integrating FL with SNNs is challenging due to time-varying channel conditions and data distributions. In addition, existing multi-width SNN training algorithms are sensitive to the data distributions across devices, which makes SNN ill-suited for FL. Motivated by this, we propose a communication and energy-efficient SNN-based FL (named *SlimFL*) that jointly utilizes *superposition coding (SC)* for global model aggregation and *superposition training (ST)* for updating local models. By applying SC, SlimFL exchanges the superposition of multiple-width configurations decoded as many times as possible for a given communication throughput. Leveraging ST, SlimFL aligns the forward propagation of different width configurations while avoiding inter-width interference during backpropagation. We formally prove the convergence of SlimFL. The result reveals that SlimFL is not only communication-efficient but also deals with non-IID data distributions and poor channel conditions, which is also corroborated by data-intensive simulations.**

Manuscript received 2 February 2022; revised 30 September 2022; accepted 18 December 2022; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor G. Joshi. Date of publication 2 January 2023; date of current version 19 December 2023. This work was supported in part by the National Research Foundation of Korea (NRF-Korea) under Grant 2022R1A2C2004869 and in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant through the Korea Government [Ministry of Science and Information and Communications Technology (MSIT)], Intelligent 6G Wireless Access System, under Grant 2021-0-00467. Preliminary version of this paper was accepted to IEEE Conference on Computer Communications (INFOCOM), May 2022 [DOI: 10.1109/INFOCOM48880.2022.9796733]. *(Won Joon Yun, Yunseok Kwak, and Hankyul Baek are co-first authors.) (Corresponding authors: Soyi Jung; Jihong Park; Joongheon Kim.)*

Won Joon Yun, Yunseok Kwak, Hankyul Baek, and Joongheon Kim are with the School of Electrical Engineering, Korea University, Seoul 02841, South Korea (e-mail: ywjoon95@korea.ac.kr; rhkrdbstjr0@korea.ac.kr; 67back@korea.ac.kr; joongheon@korea.ac.kr).

Soyi Jung is with the Department of Electrical and Computer Engineering, Ajou University, Suwon 16499, South Korea (e-mail: sjung@ajou.ac.kr).

Mingyue Ji is with the Department of Electrical and Computer Engineering, The University of Utah, Salt Lake City, UT 84112 USA (e-mail: mingyue.ji@utah.edu).

Mehdi Bennis is with the Centre for Wireless Communications, University of Oulu, 90014 Oulu, Finland (e-mail: mehdi.bennis@oulu.fi).

Jihong Park is with the School of Information Technology, Deakin University, Geelong, VIC 3220, Australia (e-mail: jihong.park@deakin.edu.au).

Digital Object Identifier 10.1109/TNET.2022.3231864

***Index Terms*— Federated learning, heterogeneous devices, slimmable neural network.**

I. INTRODUCTION

A. Background and Motivation

RECENT advances in machine learning (ML), and hardware technologies have pushed deep learning down from cloud servers to edge devices such as phones, cars, and the Internet of things (IoT) devices [2], [3]. These edge devices collectively constitute a source of ever-growing big data, so are indispensable for training high-quality machine learning models. Meanwhile, each edge device stores only a tiny fraction of the big data, which is often privacy-sensitive (e.g., navigation history, e-Health wearable records, and surveillance camera photos). To exploit these opportunities and address the challenges induced by edge devices, federated learning (FL) is a promising solution that allows edge devices to train a global model by exchanging locally trained models instead of raw data [4], [5], [6], [7].

At its core, FL operations rest on repeatedly constructing a global model averaged over the local models, which is downloaded by each device to replace its local model with the new global model. By design, FL necessitates the use of a single global model for all devices. This requirement restricts the scalability and accuracy of FL, particularly when scaling to the sheer number of edge devices, which have non-identical memory resources and energy budgets not to be mentioned as different communication channel conditions. Therefore, the use of a single large model may be suited for only a few devices, whereas the use of a tiny model should be accompanied by compromising accuracy.

In light of the aforementioned issues in communication-energy efficiencies and scalability, the recently proposed slimmable neural network (SNN) architectures have great potential in that an SNN can adjust its model width in accordance with its available energy budget or task difficulty. Inspired from this, in this article, we propose an *SNN-based FL algorithm* that makes use of *superposition coding (SC)* and *successive decoding (SD)*, coined *slimmable FL (SlimFL)*. To illustrate the effectiveness of SlimFL, consider an SNN with two width configurations, as seen in Fig. 1. Each device uploads its local updates during the uplink to the server after jointly encoding

1558-2566 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

the left-half (LH) and right-half (RH) of its local SNN model and assigning distinct transmission power levels, i.e., SC [8]. The server then makes an attempt to decode the LH. If the LH is successfully decoded, the server attempts to decode the RH consecutively, i.e., SD, also known as successive interference cancellation (SIC). As a result, when the device-server channel throughput is poor, the server decodes only the LH of the uploaded model, yielding a model with a *half-width (0.5x) width*. When the channel has a high throughput, the server may decode both LH and RH and combine them to produce the *full-width model (1.0x)*. As a result, the server creates a global model superimposing the decoded 0.5x and 1.0x local models, which each device downloads. The device substitutes the downloaded global model for its local model and repeats the preceding step until convergence. In addition, the same technique may be extended to the server-to-device downlink.

SlimFL's efficacy is contingent upon achieving synergy between several width configurations, namely 0.5x and 1.0x models, which is a difficult task. The existing FL algorithm does not focus on synergetic of 0.5x and 1.0x. In addition, since the local model has various width configurations, training them may conflict with one another. To the best of our knowledge, FL with different width configurations have not been proposed, yet. Existing SNN architectures and training techniques are designed for standalone learning and so are unsuitable for SlimFL, especially when data distributions are not independent and identical (non-IID). To address these issues in SlimFL, we provide a new SNN architecture and training method called *superposition training (ST)*, as well as a study of SlimFL's convergence and efficiency.

B. Contributions

The major contributions of this paper are summarized as follows.

- We first propose an FL framework for SNNs, SlimFL (see Fig. 1 and **Algorithm 4**), which uses SC for enhancing communication efficiency under time-varying wireless channels with limited bandwidth.
- We develop a local SNN training method for SlimFL, ST (see **Algorithm 3**), which minimizes excessive inter-width interference and achieves fast convergence with good accuracy while being agnostic to data distribution.
- We prove the convergence of SlimFL (see **Theorem 1**). Numerical results show the benefits of SlimFL in terms of channel quality and data distributions and characterize the optimum transmit power allocation for SC (see **Proposition 1**), and the optimal ST ratio allocation (see **Proposition 2**).
- We verify our analysis via simulations, indicating that SlimFL achieves higher accuracy and lower communication costs than vanilla FL (i.e., FedAvg) under poor channel conditions and non-IID data distributions.

In our previous work [1], we only described the brief idea of SlimFL and provided the proof sketches of convergence. In this work, we delineate the detailed operations and motivation of SlimFL, while elaborating on the full derivation steps of the convergence proofs. To further advocate the feasibility of SlimFL under various scenarios, we numerically study the impacts of different channel models, datasets, and local training capabilities in Sec. VI-D.

TABLE I
LIST OF NOTATIONS

Symbol	Description
K	The number of devices
T	Total iteration step
S	The number of width configurations in SNN
θ^G	Parameter of global model
θ^k	Parameter of k-th local model
Ξ_i	Binary mask to extract model parameter of i-th model
Ξ	Binary mask to extract model parameter of LH segment
Ξ^{-1}	Binary mask to extract model parameter of RH segment
H	Set of successfully decoded LH segment
F	Set of successfully decoded RH segment
n_{L}	The number of successfully decoded LH segments
n_{R}	The number of successfully decoded RH segments
p_i	Decoding success probability of i-th message
$\boldsymbol{Z}$	Entire dataset
ζ_t^k	Local data sampled from k-th device at t
w_i	Ratio of superpositioned training for updating i-th model
λ	Power allocation ratio
η_t	Learning rate of t
$\odot$	Element-wise multiplication
$M(\theta_t^k, \zeta_t^k)$	Logits from feed-forwarding ζ_t^k to k-th SNN
$y(\zeta_t^k)$	Ground truth of ζ_t^k
γ	Signal-to-interference-plus-noise ratio (SINR)
σ^2	Noise power
d	Distance between the local device and the server
β	Pathloss exponent
P	Transmit power
P_I	Interference power
u	Code rate
χ	Small-scale fading
R	Received throughput

C. Organization

The rest of this paper is organized as follows. Sec. II describes previous research results of FL and background knowledge of superposition coding and successive decoding. Sec. III presents the proposed SNN model and its training method (i.e., SUSTrain), and Sec. IV designs SlimFL using superposition coding, and successive decoding, Sec. V shows the convergence analysis on SlimFL. Sec. VI presents the simulation-based performance evaluation and its result regarding SlimFL. Finally, Sec. VII concludes this paper.

The notations used in this paper are listed in Table I.

II. Related Work

A. Energy-Constrained Federated Learning

In FL, each edge device trains its model with its own data. After training, all parameters of each local model are transmitted to the server (e.g., cloud, fog, or edge). The server reconstructs the global model by aggregating the local model parameters. FL is privacy-preserving because the device does not transmit its own data to the server. FL has advantages regarding communication/computing costs

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

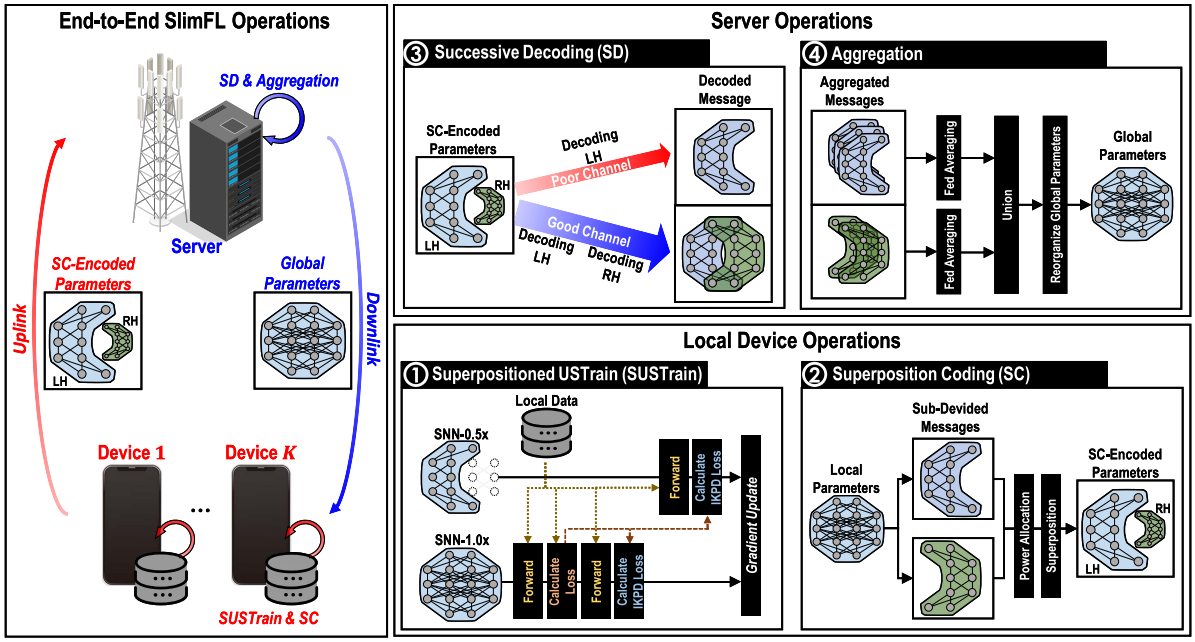


Fig. 1. A schematic illustration: SlimFL consists of four processes. On the local side, the local SNN is trained with SUSTrain. After training, the local SNN is coded with SC. The local devices transmit SC-encoded local parameters to the server. On the server side, the server successively decodes SC-encoded parameters. The server aggregates the decoded parameters and reconstructs the global parameters.

as well as privacy-preserving. For example, *Federated Averaging* (FedAvg) reduces 10-100x of communication costs compared to the existing distributed machine learning algorithm [4], [9]. In addition, FL with local batch normalization (FedBN [10]), FL with a generalization and re-parametrization (FedProx [11]), or adaptative aggregation (FedOpt [12]) improves the speed of convergence of the FL regime, which means reducing more communication/computing costs through training algorithms. However, the reduced communication/computing cost is non-negligible, because the communication cost is proportional to the model size. To cope with this problem, one approach is to optimize resource allocation [13], [14]. The other approach is split learning [15], which reduces communication costs by transmitting smashed data rather than model parameters. The other option is to investigate deep learning technique, *e.g.,* model pruning to satisfy varying on-device energy and memory limits [16] or to distill the knowledge of a big trained model into a small empty model through knowledge distillation (KD) [17], but this requires extra training operations. Alternatively, one may alter the width and/or depth of a trained model to match the resource needs. After training, depth-controlled neural networks [18] and adaptive neural networks [19] can modify their depths, while SNNs alter their widths [20]. According to studies [20], [21], [22], many SNN architectures and their algorithms are proposed. In this paper, we leverage width-controllable SNNs, and develop its FL version, SlimFL. Such an extension is non-trivial and entails several design issues, such as local SNN training algorithms, aggregating segment prioritization.

B. Superposition Coding & Successive Decoding

Non-orthogonal multiple access (NOMA) often outperforms orthogonal multiple access (OMA) in terms of sum rate and outage probability [23], [24], [25]. While uplink NOMA leverages distance and/or power differences to support more devices [26], [27], downlink NOMA commonly focuses on simultaneously broadcasting multiple signals with different priorities, such that the higher priority signal is more likely to be successfully decoded. Namely, the base station in downlink NOMA superimposes distinct data signals over the same radio block (*i.e.*, SC) by assigning different transmit power levels [8]. The mobile user decodes the SC-encoded signal through successive interference cancellation that decodes the strongest signal first, followed by decoding the next strongest signal after removing the decoded signal (*i.e.*, SD) [28].

Inspired from downlink NOMA, in this paper we apply SC and SD for the uplink operations of SlimFL. Precisely, SlimFL prioritizes an SNN's LH to obtain the 0.5x model across inferior channels. SlimFL is capable of decoding the RH of the SNN only under favorable the channel circumstances, yielding the 1.0x model by merging both LH and RH. As a result, SlimFL provides consistent convergence even in the presence of weak channels.

III. SNN Model Architecture and Training

Currently, available SNN architectures and training algorithms are optimized for standalone learning [20]. This section presents SNN architecture for SlimFL, and its local training.

A. Operations of Slimmable Neural Network

Lightweight deep learning techniques have been studied widely. Thus many complicated models are distributed to mobile devices via quantization, e.g., TensorRT [29]. Nevertheless, training models in mobile devices still requires computational cost as same as the existing methods. We utilize SNNs as both devices and server models. The key idea of SNN is to reduce floating point operations per second (FLOPS)

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: SlimTrain [21]

Define *switchable width list* for slimmable network M, for example, $[0.25, 0.5, 0.75, 1.0]\times$.
Initialize shared convolutions and fully-connected layers for slimmable network M.
Initialize independent batch normalization parameters for each *width* in *switchable width list*.
for $i = 1, \ldots, n_{iters}$ **do**
  Get next mini-batch of data x and label y.
  Clear gradients of parameters, $optimizer.zero_grad()$.
  for *width in switchable width list* **do**
    Switch the batch normalization parameters of current width on network M.
    Execute sub-network M' at current width, $\hat{y} = M'(x)$.
    Compute loss, $loss = criterion(\hat{y}, y)$.
    Compute gradients, $loss.backward()$.
  end
  Update parameters, $optimizer.step()$.
end

Algorithm 2: USTrain [20]

Define *width range*, for example, $[0.25, 0.5, 0.75, 1.0]$x.
Define *n* as the number of sampled widths per training iteration, for example, $n = 4$.
Initialize training settings of shared network M.
for $(t = 1, \ldots, T_{iters})$ **do**
  Get the next mini-batch of data x and label y).
  Clear gradients, $optimizer.zero_grad()$.
  Execute full-network, $y' = M(x)$.
  Compute loss, $loss = criterion(y', y)$.
  Accumulate gradients, $loss.backward()$.
  Stop gradients of y' as label, $y' = y'.detach()$.
  Add smallest width to *width samples*.
  for *width in width samples* **do**
    Execute sub-network at *width*, $\hat{y} = M'(x)$.
    Compute loss, $loss = criterion(\hat{y}, y')$.
    Accumulate gradients, $loss.backward()$.
  end
  Update parameters, $optimizer.step()$.
end

Algorithm 3: Superposition Training (SUSTrain)

Initialize train parameter $\Theta = \{\theta^1, \cdots, \theta^k, \cdots, \theta^K, \theta^G\}$,
Initialize local dataset $\boldsymbol{Z} = \{Z_1, \cdots, Z_k, \cdots, Z_K\}$ with Dirichlet distribution
Initialize learning rate $\eta_t \leftarrow \eta_0$
Further Constraints: $\hat{F}^1(\cdot) = F^1(\cdot)$ ▷ Discuss in Sec. V
for $t = 1, \cdots, T$ **do**
  for $k = 1, \cdots, K$ **do**
    Initialize gradients of the model optimizer as 0.
    Sample batch ζ_t^k from Z_k.
    Execute full-network $M(\theta_t^k, \zeta_t^k)$.
    Compute loss, $loss \leftarrow F^k(\theta_t^k, \zeta_t^k)$.
    Accumulate gradients, $loss.backward()$.
    Execute full-network $M(\theta_t^k, \zeta_t^k)$.
    Stop gradients of $M(\theta_t^k, \zeta_t^k)$ as label.
    for $i = 1, \cdots, S-1$ **do**
      Execute and calculate loss $\hat{F}^k(\theta_t^k \odot \Xi_i, \zeta_t^k)$
      $loss \leftarrow loss + w_i \hat{F}^k(\theta_t^k \odot \Xi_i, \zeta_t^k)$.
    end
    Calculate gradient of $loss$.
    Update model parameters. ▷ Eq. (3)
  end
end

by activating the selected weights. We compare the existing NN operation to SNN's. Suppose that input and output pairs are denoted as $\mathbf{w}$ and $\mathbf{v}$, respectively. The linear operation is written as follows:

$$\mathbf{v} = \mathbf{W} \cdot \mathbf{w} + \mathbf{b}, \tag{1}$$

where $\mathbf{W}$, $\mathbf{b}$ denote $|\mathbf{v}|\times|\mathbf{w}|$ sized matrix and $|\mathbf{v}|$ sized vector, respectively. Compared to linear operation, the slimmable linear operation utilizes the shrink ratio is denoted as $r \in \mathbb{R}[0, 1]$ to reduce FLOPS. In the linear operation, the output dimension is reduced by zero-masking $\mathbf{W}$ and $\mathbf{b}$. The following masked weights and biases are obtained as follows:

$$\mathbf{W}_{\text{masked},r} = \mathsf{Mask}(\mathbf{W}, r), \quad \mathbf{b}_{\text{masked},r} = \mathsf{Mask}(\mathbf{b}, r), \tag{2}$$

where $\mathsf{Mask}(\mathbf{W}, r)$ and $\mathsf{Mask}(\mathbf{b}, r)$ reduces the size by tensor product into $\lceil|\mathbf{v}| \cdot r\rceil \times |\mathbf{w}|$ and $\lceil|\mathbf{v}| \cdot r\rceil$, respectively. The mobile device can reduce the computational complexity in both training and interference phases up to $\mathcal{O}(r^2)$.

Hereafter, we consider two width configurations with the ratio $r = 0.5$ and $r = 1.0$ (i.e., 0.5x or 1.0x). At the t-th iteration, the SNN model has the parameters θ_t^k with two width configurations: 0.5x width configuration $\theta_t^k \odot \Xi_1$ and 1.0x width configuration $\theta_t^k \odot \Xi_2$ $(= \theta_t^k)$, where $\odot$ is the element-wise product and Ξ_i represents a binary mask for extracting the parameters of i-th width configuration.

B. Superposition SNN Training

Training a multi-width SNN is arduous. The SNN back-propagation (BP) is warped due to gradient interference of multi-width. For example, the BP of 0.5x width configurations interferences the BP of 1.0x's and vice versa. This inter-width interference hinders not just inference accuracy but also training convergence. SlimTrain (see **Algorithm 1**), the first SNN training algorithm introduced in [21], partially ameliorates such inter-width interference by training alternative width configurations in descending order of size. While adhering to the sample principle, the authors of [20] have proposed a state-of-the-art SNN training technique called universal SNN (USTrain).

USTrain proposes the sandwich rule with the inplace knowledge distillation (IPKD), which are two notable techniques for training an SNN. To describe two notable techniques, the 1.0x

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

model becomes a teacher guiding its sub-width models via knowledge distillation. By nature, it gives more benefits under a larger SNN (i.e., a better teacher) that has more sub-width configurations (i.e., more students). The IPKD encourages each sub-width (i.e., student) to provide a softmax output (i.e., logit) to that of the full-width (i.e., teacher), so that their overlapping BP gradients become less dissimilar, hence minimizing the inter-width interference. At its core, the inplace distillation and the sandwich rule are effective under the case where an SNN can be divided into more than two segments. The SNN architecture considered in this paper consists of only the LH and RH segments, making the USTrain unfit for our case. Moreover, during updating the local parameters of slimmable model, some of the parameters composing small model needs partial gradient calculation of big model.

While effective in standalone learning, in SlimFL with wireless connectivity, not all multi-width configurations are exchanged due to insufficient communication throughput. In other words, they exchanged width configurations are aggregated across devices, diluting the effectiveness of BN. In our experiments, we even observed training convergence failures due to BN. Furthermore, managing multiple BN layers not only consumes additional memory costs but also entails a high computing overhead.

To resolve this problem, we propose *superpositioned USTrain (SUSTrain)* algorithm. SUSTrain consists of two processes. First, all the forward propagation losses (FP) are holden. Then, all the width configurations are concurrently updated with the superpositioned gradients. With SUSTrain, a sub-width configuration (i.e., student) is trained using IPKD without the logit mismatch with its full-width configuration's logit (i.e., teacher's logit), while the full-width configuration is simultaneously trained using the ground truth.

In this paper, we consider that all devices have SNN with two-width configurations (i.e., one teacher and one student). Hereafter, we generalize the local SNN update rule for the device k described as follows:

$$\theta_{t+1}^k = \theta_t^k - \eta_t \left[w_1 \nabla \hat{F}^k(\theta_t^k \odot \Xi_1, \zeta_t^k) + w_2 \nabla F^k(\theta_t^k \odot \Xi_2, \zeta_t^k) \right], \tag{3}$$

where $w_1 + w_2 = 1$ and $w_1, w_2 > 0$. The operator $\odot$ implies element-wise multiplication for extracting the parameters which is allocated to the SNN width configuration. The term $\eta_t > 0$ is a learning rate, and ζ_t^k implies a stochastic input realization. The function $F^k(\theta_t^k \odot \Xi_i, \zeta_t^k)$ is the cross-entropy between the ground truth $y(\zeta_t^k)$, whereas the IPKD function $\hat{F}^k(\theta_t^k \odot \Xi_i, \zeta_t^k)$ is the cross-entropy between the logit $M(\theta_t^k, \zeta_t^k)$ of the full-width configuration and the logit $M(\theta_t^k \odot \Xi_i, \zeta_t^k)$ of the i-th width configuration. The details of SUSTrain are in **Algorithm 3**.

IV. Global Model Aggregation With Superposition Coding & Successive Decoding

A. Superposition Coding & Successive Decoding

1) Wireless Systems: Consider a single base station equipped with a server. The server is associated with K mobile devices at equal distance d, e.g., uniformly distributed around a circle centered at the server with a radius of d. Unless otherwise specified, hereafter we focus only on uplink communications. In the uplink, each device aims to upload S messages corresponding to S width configurations. For ease of explanation, we consider frequency division duplex (FDD) and orthogonal frequency division multiplexing (OFDM), which can be replaced with other duplexing and multiplexing schemes such as time division duplex (TDD) and time division multiplexing (TDM) [30] with minor modifications. For simplicity without loss of generality, we consider that the bandwidth is orthogonally and equally allocated to each device, thereby ignoring inter-user interference. Notwithstanding, we still consider the interference due to SC and SD operations. For mathematical amenability, we assume that the interference is treated as noise, and use the Shannon's capacity formula with the Gaussian codebook, i.e., the use of optimal source and channel coding, which are widely used assumptions for analysis [28].

2) Superposition Coding (SC): Recall that each mobile device k stores a local model having S width configurations. When $S = 2$, the model is equally split as the LH and the RH segments, and the half-width model (0.5x) is the LH while the entire model (1.0x) is obtained by combining the LH and the RH. Generalizing this, we can treat S width configurations as S messages, and concurrently transmit them via SC [8]. To this end, consider that P_i transmit power is allocated to the i-th message out of the total power budget P, i.e., $P = \sum_{i=1}^{S} P_i, \forall i \in [1, S]$. Consider a mobile device k transmitting a signal $\mathbf{x}_k$ to the server over the same radio resource block, which is as:

$$\mathbf{x}_k = \sum\nolimits_{i=1}^{S} \mathbf{s}_{k,i}, \tag{4}$$

where $\mathbf{s}_{k,i} \in \mathcal{S}_k$ denotes the symbol i of mobile device k, and $\mathcal{S}_k$ is the Gaussian codebook for device k, with $\mathbb{E}[\mathbf{s}_{k,i}] = 0$ and $\mathbb{E}[|\mathbf{s}_{k,i}|^2] = P_i$, $\forall i \in [1, S], \forall k \in [1, K]$.

3) Decoding Success Probability and Successive Decoding (SD): In SD, the receiver first decodes the strongest signal. Then, it sequentially decodes the next strongest signal, after cancelling out the decoded signal while treating the rest as interference, i.e., successive interference cancellation [31]. For simplicity, assume that $P_i > P_{i'}$ for all $i' > i$. Following SD, for the device k, the server first decodes $\mathbf{s}_{k,1}$ while treating the rest $P_{k,1}^I$ as interference, followed by decoding $\mathbf{s}_{k,2}$ in the presence of the interference $P_{k,2}^I \leq P_{k,1}^I$. Each decoding becomes successful when the throughput $R_{k,i}$ at the server is no smaller than a pre-defined transmit rate $u > 0$. The throughput is given by the Shannon's capacity formula, yielding $R_{k,i} = W \log_2(1 + \gamma_{k,i})$ (bits/sec), where W is the bandwidth, and $\gamma_{k,i}$ denotes the signal-to-interference-plus-noise ratio (SINR).

Precisely, the received signal at the server, denoted by $\mathbf{y}_k$, is given by:

$$\mathbf{y}_k = \mathbf{h}_k^H \mathbf{x}_k + \mathbf{n}_k = \mathbf{h}_k^H \sum\nolimits_{i=1}^{S} \mathbf{s}_{k,i} + \mathbf{n}_k, \tag{5}$$

where $\mathbf{h}_k$ is the channel coefficient, $\mathbf{n}_k \sim \mathcal{CN}(0, \sigma_{\mathbf{h}}^2)$ represents the additive white Gaussian noise (AWGN), and $(\cdot)^H$

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

stands for the Hermitian transpose operator. According to SD, the SINR for the i-th message $s_{k,i}$ is given by:

$$\gamma_{k,i} = \frac{|\mathbf{h}_k^H \mathbf{s}_{k,i}|^2}{\sigma^2 + \sum_{i'=i+1}^{S} |\mathbf{h}_k^H \mathbf{s}_{k,i'}|^2}, \tag{6}$$

where σ^2 denotes the noise power. For ease of notation, the SINR in (6) is recast as:

$$\gamma_{k,i} = \chi_k \cdot d^{-\beta} \cdot P_i / (\sigma^2 + P_{k,i}^I), \tag{7}$$

in which $\mathbb{E}[|\mathbf{h}_k|^2] = d^{-\beta}$ and $\chi_k \sim \mathsf{Exp}(1)$ follow from the large-scale fading and Rayleigh small-scale fading, respectively. The term $P_{k,i}^I = \chi_k d^{-\beta} \hat{P}_i^I$ is the interference, where $\hat{P}_i^I \triangleq \sum_{i'=i+1}^{S} P_{i'}$ for $i \leq S-1$, and $\hat{P}_S^I = P_{k,S}^I = 0$ as there is no interference for the last message.

In SD, the decoding success of the i-th message implies that not only the i-th but also its all subsequent decoding successes; in other words, $R_{k,1} > u, R_{k,2} > u, \dots R_{k,i} > u$ or equivalently $\max\{R_{k,1}, R_{k,2}, \dots R_{k,i}\} > u$. Recall that S messages encoded as $\mathbf{x}_k$ experience the same channel. Then, the decoding success probability p_i of the i-th message is given as follows:

$$p_i = \Pr\left(\chi_k \geq \max\left\{\frac{c}{P_1/u' - \hat{P}_1^I}, \cdots, \frac{c}{P_i/u' - \hat{P}_i^I}\right\}\right) \tag{8}$$

$$= \exp\left(-\max\left\{\frac{c}{P_1/u' - \hat{P}_1^I}, \cdots, \frac{c}{P_i/u' - \hat{P}_i^I}\right\}\right), \tag{9}$$

where $c = \sigma^2 d^\beta$, $u' = 2^{\frac{u}{W}} - 1$, and the last step follows from the complementary cumulative distribution function (CCDF) of χ_k.

B. SlimFL Operations

We discuss SlimFL and global model aggregation in further detail. SlimFL is denoted by the symbols shown in Table I. **Algorithm 4** describes the fundamental SlimFL operations. The network consists of K devices that are linked through wireless connections to a parameter server. Each device uses SC whilst communicating with the server, whereas the server uses SD. To be more descriptive, k-th device has a local dataset $Z^k \in \boldsymbol{Z}$ and an SNN parameter θ^k with two width configurations. Across devices, the global data $\boldsymbol{Z}$ may be IID or non-IID. Each SNN θ^k is subdivided into an LH $\theta^k \odot \Xi$ and a RH segment $\theta^k \odot \Xi^{-1}$, where $\Xi = \Xi_1$ and $\Xi^{-1} = \Xi_2 - \Xi_1$. Superposition training is used to train the k-th local device (lines 4–9), which is expressed as (3). The local device transmits to the server the SC-encoded local model θ^k. Each local device transmits two messages (i.e., LH and RH segments), each with a distinct transmission power P_1 and P_2 relative to $P_1 \gg P_2$. The uniform random variable $\rho \in [0, 1]$ is sampled for user k as ρ_k. The sampled ρ_k and the decoding success probability p_1, p_2 are used as a criterion for whether LH and RH are successfully decoded. It is identical to calculate (9). In accordance with (9), after reception, the server can successively decode using SD and obtain: (i) a 0.5x model if $\chi \geq c/(P_1/u' - P_2)$ (lines 15–17) is satisfied; (ii) 1.0x model if the channel fading gain satisfies $\chi \geq \max\{c/(P_1/u' - P_2), c/(P_2/u')\}$ (lines 18–20); and (iii) otherwise it obtains no model. As a result, the RH segments from F of devices and the LH segments from $\mathsf{H} \cup \mathsf{F}$ of devices are combined by the server.

Algorithm 4: SlimFL With SC & SD

1 Initialize train parameters $\Theta = \{\theta^1, \cdots, \theta^k, \cdots, \theta^K, \theta^G\}$.
2 Split dataset $\boldsymbol{Z}$ into K datasets $\boldsymbol{Z} = \{Z_1, \cdots, Z_k, \cdots, Z_K\}$.
3 **while** *Training* **do**
4 `//Local Model Training (Algorithm 1)`
5 **for** $k = 1, \cdots, K$ **do**
6 **for** ζ_k *in* Z_k **do**
7 Update local model parameter θ^k ▷ Eq. (3)
8 **end**
9 **end**
10 `//SC&SD-based Server Aggregation (Uplink)`
11 **if** *Aggregation Period* **then**
12 $n_{\mathsf{L}} = |\mathsf{H} \cup \mathsf{F}| \leftarrow 0, n_{\mathsf{R}} = |\mathsf{F}| \leftarrow 0,$
13 $\mathsf{H} \leftarrow \varnothing, \mathsf{F} \leftarrow \varnothing$
14 **for** $k = 1, \cdots, K$ **do**
15 $\rho_k \leftarrow rand(1)$
16 **if** $p_2 \leq \rho_k < p_1$ **then**
17 $\mathsf{H} \leftarrow \mathsf{H} \cup k$, $n_{\mathsf{L}} \leftarrow n_{\mathsf{L}} + 1$
18 **end**
19 **if** $\rho_k \geq p_2$ **then**
20 $\mathsf{F} \leftarrow \mathsf{F} \cup k$, $n_{\mathsf{L}} \leftarrow n_{\mathsf{L}} + 1$, $n_{\mathsf{R}} \leftarrow n_{\mathsf{R}} + 1$
21 **end**
22 **end**
23 ▷ **Case1.** $n_{\mathsf{L}} > 0$**,** $n_{\mathsf{R}} > 0$
24 $\theta^G \leftarrow \frac{1}{|\mathsf{H} \cup \mathsf{F}|} \sum_{k \in \mathsf{H} \cup \mathsf{F}} \theta^k \odot \Xi + \frac{1}{|\mathsf{F}|} \sum_{k \in \mathsf{F}} \theta^k \odot \Xi^{-1}$
25 ▷ **Case2.** $n_{\mathsf{L}} > 0$**,** $n_{\mathsf{R}} = 0$
26 $\theta^G \leftarrow \frac{1}{n_{\mathsf{L}}} \sum_{k \in \mathsf{H}} (\theta^k \odot \Xi)$
27 ▷ **Case3.** $n_{\mathsf{L}} = n_{\mathsf{R}} = 0$
28 **Skip aggregation**
29 **end**
30 `//Local Update (Downlink)`
31 **for** $n = 1, \cdots, K$ **do**
32 $\theta^k \leftarrow \theta^G$
33 **end**
34 **end**

V. SlimFL Convergence Analysis

A. Assumptions

In order to analyze the convergence rate of SlimFL, the following assumptions are considered.

1) Regardless of SC or SD, downlink decoding is always successful (**Algorithm 4**, lines 29–32). The fact that the server (e.g., a base station) has a far higher broadcast power than the uplink power contributes to this.
2) We assume that K is big enough that $|\mathsf{H} \cup \mathsf{F}| \approx Kp_1$ and $|\mathsf{F}| \approx Kp_2$, where p_1 and p_2 are the LH and RH segment decoding success probability, respectively, provided in (9). As a result, during the t-th communication

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

cycle, the server builds the following global model θ_t^G:

$$\theta_t^G \leftarrow \frac{1}{Kp_1}\sum\nolimits_{k\in \mathsf{H}\cup\mathsf{F}} \theta_t^k \odot \Xi + \frac{1}{Kp_2}\sum\nolimits_{k\in\mathsf{F}} \theta_t^k \odot \Xi^{-1}. \tag{10}$$

3) One communication round is assumed per local iteration or mathematically tractability. According to [32], FedAvg does not guarantee that a number of local iterations is proportional to the performance. Since the number of local iterations at each communication round is 1, superscript G is omitted, resulting in $\theta_t = \theta_t^G$. This assumption will be further discussed in Sec VI-E.3.

Based on these assumptions, the convergence analysis of SlimFL is mathematically tractable as discussed in Sec. V-B.

B. Convergence Analysis

This paper analyzes the convergence of SlimFL in non-IID data distribution. We follow the fundamental derivation techniques utilized [33], [34] for FedAvg. One significant challenge in the convergence analysis for SlimFL is due to the local model updates in (3) and the global model aggregation in (10) include sophisticated binary masks owing to the SNN architecture as well as SC and SD. As a result, unlike FedAvg, whose global objective function is the weighted average of local loss functions $\{F^k(\theta_t^k)\}$, i.e., empirical risk, SlimFL's objective function $F(\theta_t)$ is unknown. Alternatively, we define $F(\theta_t)$ in terms of its gradient $f_t = \nabla F(\theta_t)$, which can be obtained using SlimFL's local and global operations, as discussed below. After a downlink, the device k replaces its local model with the downloaded global model, represented by $\theta_t^k \leftarrow \theta_t$. The device then changes the local model as:

$$\theta_{t+1}^k \leftarrow \theta_t - \eta_t g_t^k, \tag{11}$$

where $g_t^k = \sum_{i=1}^2 w_i \nabla F^k(\theta_t \odot \Xi_i, \zeta_t^k)$ follows from (3). We assume that the student's soft goal may be approximated by the student's hard objective, i.e.,

$$\hat{F}^k(\theta_t \odot \Xi_i, \zeta_t^k) \approx F^k(\theta_t \odot \Xi_i, \zeta_t^k). \tag{12}$$

Next, after the uplink, the server aggregates the updated local models to form the global model θ_{t+1}. By applying (11) to (10), the resulting global model is as follows:

$$\begin{aligned}\theta_{t+1} &= \frac{1}{Kp_1}\sum_{k\in\mathsf{H}\cup\mathsf{F}} (\theta_t - \eta_t g_t^k)\odot\Xi + \frac{1}{Kp_2}\sum_{k\in\mathsf{F}} (\theta_t - \eta_t g_t^k)\odot\Xi^{-1} \\ &= \theta_t - \eta_t \underbrace{\Big(\frac{1}{Kp_1}\sum_{k\in\mathsf{H}\cup\mathsf{F}} g_t^k \odot \Xi + \frac{1}{Kp_2}\sum_{k\in\mathsf{F}} g_t^k \odot \Xi^{-1}\Big)}_{:=f_t},\end{aligned} \tag{13}$$

resulting in f_t in (13), which characterizes $F(\theta_t)$. In (13), the last step can be obtained from $|\mathsf{H}\cup\mathsf{F}| = Kp_1$, $|\mathsf{F}| = Kp_2$, and $\theta_t = \theta_t \odot (\Xi + \Xi^{-1})$.

Hereafter we use the bar notation $\bar{\cdot}$ for the value averaged over $\{\zeta_t^k\}$, and * for indicating the optimum. For the functions F and $\{F^k\}$, we analyze the following commonly used assumptions to prove convergence of FedAvg under non-IID data distributions [33], [35]. According to [36], following assumptions are justifiable in SNN architectures.

Assumption 1: **(L-smoothness)** F and $\{F^k\}$ are L-smooth, i.e.,

$$F^k(\theta_v) \le F^k(\theta_w) + (\theta_v - \theta_w)^T \nabla F^k(\theta_w) + \frac{L}{2}\|\theta_v - \theta_w\|^2 \tag{14}$$

for all $v, w > 0$.

Assumption 2: **(μ-strong convexity)** F and $\{F^k\}$ are μ-strong convex: i.e.,

$$F^k(\theta_v) \ge F^k(\theta_w) + (\theta_v - \theta_w)^T \nabla F^k(\theta_w) + \frac{\mu}{2}\|\theta_v - \theta_w\|^2 \tag{15}$$

for all $v, w > 0$.

Since linear sum preserves strong convexity and smoothness, $\tilde{F}^k$ also has the μ-strong convexity and L-smoothness. In addition, we also assume our global model has same properties.

Assumption 3: **(Bounded local gradient variance)** The variance of the local gradient $\nabla F^k(\theta^k, \zeta_t^k)$ is bounded within Z_k, which is given as

$$\mathbb{E}[\|\nabla F^k(\theta^k, \zeta_t^k) - \nabla \bar{F}^k(\theta)\|^2] \le \sigma_k^2. \tag{16}$$

Inspired by [34], we define a factor that measures the non-IIDness of $\boldsymbol{Z}$ as:

$$\delta = \frac{1}{K}\sum\nolimits_{k=1}^K \sigma_k^2. \tag{17}$$

Indeed, the variance (over k) of the local gradient variance (over Z_k) is defined as:

$$\bar{\sigma} \triangleq \frac{1}{K}\sum\nolimits_{k=1}^K \left(\sigma_k - \frac{1}{K}\sum\nolimits_{k=1}^K \sigma_k\right)^2. \tag{18}$$

This characterizes the data distributions over devices, and so does δ without loss of generality.

To prove the convergence of SlimFL, we derive the following two lemmas as referred to Appendix I-A, I-B.

Lemma 1: **(Bounded global gradient variance)** Under Assumption 3, the variance of the global gradient f_t is bounded within $\boldsymbol{Z}$, which is given as

$$\mathbb{E}\|f_t - \bar{f}_t\|^2 \le B \tag{19}$$

where $B = 4\delta(\frac{1}{p_1} + \frac{1}{p_2})\sum_{i=1}^2 w_i^2$.

Lemma 2: **(Per-round global model progress)** Under Assumptions 1 and 2 with a learning rate $\eta_t \le \frac{1}{L}$, the error between the updated global model and its optimum progress as

$$\mathbb{E}\|\theta_{t+1} - \theta^*\|^2 \le (1 - \frac{\mu\eta_t}{2})\mathbb{E}\|\theta_t - \theta^*\|^2 + \eta_t^2 B \tag{20}$$

By Lemma 1 and Lemma 2, the convergence of SlimFL is proved. The full derivation of this proof is in Appendix I-C.

Theorem 1: (**SlimFL Convergence**) Under Assumptions 1–3 with the learning rate $\eta_t = \frac{2}{\mu t + 2L - \mu}$, one has

$$\mathbb{E}[F(\theta_t)] - F^* \le \frac{L}{\mu}\cdot\frac{\mu L\Delta_1 + 2B}{\mu t + 2L - \mu}, \tag{21}$$

where $B = 4\delta(\frac{1}{p_1} + \frac{1}{p_2})\sum_{i=1}^2 w_i^2$ and $\Delta_t \triangleq \mathbb{E}\|\theta_t - \theta^*\|^2$. Therefore, $\mathbb{E}[F(\theta_t)]$ converges to F^* as $t \to \infty$.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

C. Several Insights of Convergence Analysis

The result of Theorem 1 exhibits several insights into SlimFL.

1) Failure Under Extremely Poor Channels: Consider a channel with very poor quality, where the server is unable to decode 1.0x models and hence collects only 0.5x models (i.e., $p_2 \approx 0$ and $p_1 > 0$). In this case, despite the optimality gap aggregating 0.5x models, the term B diverges. In these channel conditions, SC becomes inefficient, and vanilla FL with 0.5x models is preferred to SlimFL.

2) Robustness to Poor Channels: In (21), we confirm that an increasing number of 0.5x and 1.0x models aggregated (i.e., increasing p_1 and p_2) helps equally to reach the global optimal bound. As a result, aggregating 0.5x models can further alleviate 1.0x models' frequent decoding failures on poor channels.

3) Robustness to Non-IID Data: The optimality gap widens as δ (i.e., more non-IID). Contrary to vanilla FL, which benefits primarily from aggregating either 0.5x or 1.0x models, SlimFL's increased gap may be mitigated by aggregating not just 1.0x but also 0.5x models. Hence, we conclude that SlimFL is superior for non-IID data distributions and moderately poor channel conditions where $0 \ll p_1, p_2 < 1$. Vanilla FL with just 1.0x models or 0.5x models is recommended for highly excellent (i.e., $p_2 \approx 1$) or extremely bad (i.e., $p_1 \approx 0$) channel conditions, respectively. SlimFL's favorable conditions and efficacy will be validated by simulation in Sec. VI.

4) Design Principle of SlimFL: Additionally, Theorem 1 gives design principles for SC and ST as detailed in the following two propositions.

*Proposition 1 (**Optimal SC power allocations**):* Consider the SC power allocation ratio $\lambda \in (0.5, 1]$ such that $P_1 = \lambda P$ and $P_2 = (1-\lambda)P$. If $\lambda \gg \max\{0.5, cu'(1+u')/P\}$, the optimal SC power allocation ratio that minimizes the RHS of (21) is given as

$$\lambda^* = \frac{u' + \sqrt{1+u'} - 1}{u'}. \tag{22}$$

Proof: Define $D \triangleq \frac{1}{p_1} + \frac{1}{p_2}$. According to the RHS of (21), λ^* minimize D. Since $P_1 > P_2$, we have

$$D = \exp\left(-\frac{c}{\lambda P/u' - (1-\lambda)P}\right) + \exp\left(-\frac{c}{(1-\lambda)P/u'}\right). \tag{23}$$

If $\lambda \gg cu'(1+u')/P$, we can approximate both terms in D using the first-order Taylor expansion, yielding

$$D \approx 2 + \frac{c}{\lambda P/u' - (1-\lambda)P} + \frac{c}{(1-\lambda)P/u'}. \tag{24}$$

The approximated D is convex, and the first-order necessary condition gives the optimum. □

Note that the condition $\lambda \gg \max\{0.5, cu'(1+u')/P\}$ can be satisfied under sufficiently small model sizes (e.g., $t' \to 0$), wide bandwidth (e.g., $W \to \infty$), good channel conditions (e.g., $\sigma^2 \to 0$), and/or a large total transmit power budget (e.g., $P \to \infty$).

*Proposition 2 (**Optimal ST ratio**):* The optimal ST ratio that minimize the RHS of (21) are given as $w_1^* = w_2^* = 1/2$.

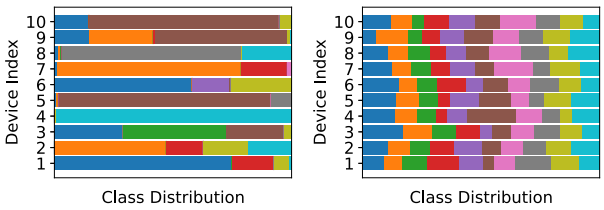


Fig. 2. An illustration of the data distributions across 10 devices for the different values of the Dirichlet concentration ratio α. Note that each color represents each class constituting the dataset.

Proof: The RHS of (21) is minimized at the minimum of $\sum_{i=1}^{2} w_i^2$. By the C-S inequality, we have

$$\sum_{i=1}^{2} w_i^2 \geq \frac{1}{2}\left(\sum_{i=1}^{2} w_i\right)^2. \tag{25}$$

The desirable result is obtained by combining the condition $\sum_{i=1}^{2} w_i = 1$ in (3) with the equality condition of the AM-GM inequality. The influence of λ^* and w_i^* will be demonstrated by simulation in the next section.

□

VI. Experiments

We investigate the effectiveness and feasibility of SlimFL corresponding to the numerical results of convergence analysis, the robustness to various channel conditions and non-IID data distributions, and computation/communication efficiency.

A. Simulation Settings

To show the effectiveness of the proposed SlimFL, we consider a classification task with the Fashion MNIST (FMNIST) image dataset [37]. The impact of other datasets such as the CIFAR-10 and MNIST datasets will be discussed in VI-E. The dataset is randomly sharded and distributed across devices. Following [38], the non-IIDness of the data distribution is modeled using a Dirichlet distribution with its concentration parameter $\alpha \in \{0.1, 1.0, 10\}$, where the higher α implies the more non-IID data distribution as illustrated in Fig 2. For numerical experiments, we adopt the light model of slimmable MobileNet, i.e., UL-MobileNet.[1] We consider a single communication round consisting of a pair of uplink and downlink transmissions for each local training iteration. The communication channels over different devices are orthogonal in both uplink and downlink. The Rayleigh fading gain χ for each channel realization follows an exponential distribution $\chi \sim \mathsf{Exp}(1)$ [28]. The corresponding communication and neural network hyperparameters are summarized in Table II. We adopt Vanilla FL as a comparison technique. Vanilla FL is widely used FL technique (i.e., FedAvg) with a fixed-width SNN model without leveraging SC nor SD. We consider three Vanilla FL schemes as listed follows:

[1]This paper utilizes *Ultra-Light MobileNet* (UL-MobileNet) under the consideration of the limited computing capacity of local.It follows MobileNet architecture [39]. UL-MobileNet consists of five convolution layers activated by the ReLU6 function, average pooling function, and linear layer.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS

Description	Value
Initial learning rate (η_0)	10^{-3}
Optimizer	Adam
Batch size	64
Distance (d)	100 [m]
Path loss exponent (β)	2.5
Bandwidth per device (W)	75 [MHz]
Uplink transmission power (P)	23 [dBm]
Central frequency	5.9 [GHz]
Noise power spectrum	-169 [dB/Hz]

TABLE III
COMPUTING COSTS AND TRANSMISSION POWER OF UL-MOBILENET

Description		1.0x	0.5x
Computation	MFLOPS / round	2.76	0.79
	# of parameters	4,586	2,293
	Bits / round	172,688	86,344
Transmission Power (P) [mW]		132.1	67.4

TABLE IV
TRANSMISSION AND COMPUTING COSTS PER COMMUNICATION ROUND

Metric	SlimFL	Vanilla FL-1.5x
Communication Cost [mW/Round]	199.5	399.1
Computation Cost [MFLOPS/Epoch]	3.56	3.56

1) *Vanilla-FL* 0.5x/1.0x: Vanilla FL 0.5x and 1.0x use only 0.5x- or 1.0x-width configurations, respectively. We compare model accuracy and the convergence of SlimFL with Vanilla FL-0.5x and 1.0x corresponding to the robustness to various channel conditions and non-IIDness in Sec. VI-B.
2) *Vanilla-FL* 1.5x: Due to the lack of width-adjustable SNNs, each device in Vanilla FL-1.5x separately runs fixed-width 0.5x and 1.0x models. Then, devices can choose the 0.5x or 1.0x model considering energy heterogeneity, which SlimFL can. In other words, Vanilla FL-1.5x operates the two FedAvg operations separately for 0.5x and 1.0x models by doubling the bandwidth, transmission power, and computing resources. By comparing SlimFL to Vanilla FL-1.5x, we investigate the difference of accuracy, received bits and energy cost.

B. Efficiency, Robustness, and Scalability of SlimFL

We carry out to analyze SlimFL's performance to Vanilla FL in situations with a variety of communication conditions and non-IID settings. To assess the efficiency of communication and computation, we first calculate computation cost for UL-MobileNet feed-forwarding [40] and communication cost per one communication round.

1) Communication Efficiency: Between ten devices and a server, the total quantity of data communicated is 205.8MBytes for SlimFL and Vanilla FL-1.0x, and 102.9MBytes for Vanilla FL-0.5x under ideal channel conditions (i.e., always successful decoding). We investigate the efficiency of communication resources in both good and bad channel conditions to validate the effectiveness of SC and SD. The experimental result for this subsection is presented in Table V, which shows that SlimFL delivers up to 3.52% fewer dropped bits than Vanilla FL-1.0x when SC and SD are being used. SlimFL enjoys the benefits mentioned above while consuming only half of the transmission power and bandwidth compared to Vanilla FL-1.5x, as presented in Table V, corroborating its communication efficiency. Because a part of the transmission power of SlimFL is allocated to 0.5x models, SlimFL decodes less than 1.0x model bits than Vanilla FL-1.0x. In return, SlimFL receives not only 1.0x models but also 0.5x models simultaneously. The additional received 0.5x models coincide with the LH parts of the 1.0x models, which improve the accuracy and convergence speed of both 0.5x and 1.0x models.

TABLE V
SUCCESSFULLY DECODED BITS OF SLIMFL, AND VANILLA FL

FL Scheme	Decoding	Channel Condition: Good	Channel Condition: Poor
SlimFL	0.5x	1.96	18.32
	1.0x	198.45	130.10
	None	5.46	57.44
Vanilla FL-0.5x	0.5x	102.21	93.87
	None	0.72	9.06
Vanilla FL-1.0x	1.0x	200.30	144.93
	None	5.56	60.96

TABLE VI
TOTAL COMPUTATION COST AND TRANSMISSION POWER OF SLIMFL AND VANILLA FL-1.5X IN VARIOUS NON-IIDNESS ($\alpha = 0.1, 1.0, 10$)

Metric	non-IIDness	SlimFL: Good	SlimFL: Poor	Vanilla FL-1.5x: Good	Vanilla FL-1.5x: Poor
Communication Cost [W]	$\alpha = 0.1$	71.0	57.3	158.8	196.8
	$\alpha = 1.0$	8.5	10.4	15.8	36.7
	$\alpha = 10$	3.03	3.51	10.2	25.4
Computation Cost [GFLOPS]	$\alpha = 0.1$	1.27	1.02	1.88	2.41
	$\alpha = 1.0$	0.15	0.18	0.22	0.51
	$\alpha = 10$	0.05	0.06	0.14	0.35

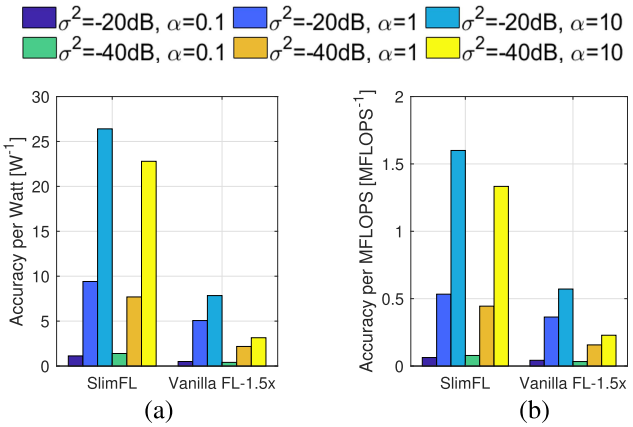


Fig. 3. Energy efficiency comparison between SlimFL and Vanilla FL-1.5x, in terms of (a) accuracy per unit communication energy and (b) accuracy per accuracy per unit computing energy.

2) Energy Efficiency: Thus far, we have evaluated SlimFL's performance after training using 1,000 epochs. We estimate the energy consumption till convergence in this section.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
ACCURACY UNDER DIFFERENT CHANNEL CONDITIONS AND α

Method	Top-1 Accuracy (%)					
	Good			Poor		
	$\alpha = 0.1$	$\alpha = 1$	$\alpha = 10$	$\alpha = 0.1$	$\alpha = 1$	$\alpha = 10$
SlimFL-0.5x	54 ± 2.2	83 ± 1.0	85 ± 1.0	56 ± 2.4	82 ± 1.7	85 ± 1.1
SlimFL-1.0x	59 ± 2.3	85 ± 1.1	87 ± 1.1	65 ± 2.9	84 ± 1.4	87 ± 0.9
Vanilla FL-0.5x	45 ± 5.9	84 ± 1.1	85 ± 1.0	39 ± 8.3	83 ± 1.2	85 ± 0.9
Vanilla FL-1.0x	69 ± 5.8	85 ± 4.0	86 ± 4.3	55 ± 9.2	80 ± 6.0	82 ± 4.7

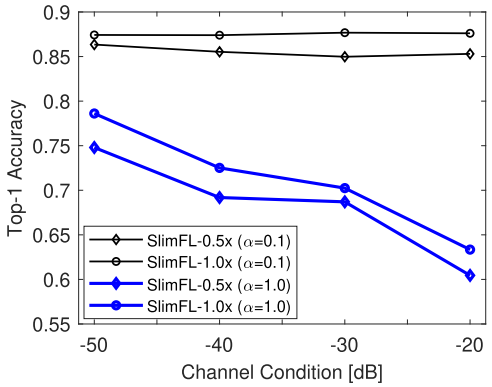


Fig. 4. Test accuracy in various channel conditions (e.g., -50dB, -40dB, -30dB and -20dB) with various non-IIDness (e.g., $\alpha = 0.1$ and $\alpha = 10$).

Numerical convergence in training is defined as the point at which the standard deviation (std) of test accuracy falls below a predefined threshold and the lowest test accuracy exceeds the average test accuracy in 100 consecutive rounds. To assess model convergence, we establish the reference values for the mean μ_{Ref} as 80% and σ_{Ref} as 7.25%, respectively. Convergence occurs when the average Top-1 accuracy for 100 consecutive epochs is greater than μ_{Ref} and the average std is less than σ_{Ref}. With this convergence criterion, Table VI compares the overall energy costs of SlimFL and Vanilla FL-1.5x until convergence, based on the communication and processing energy costs per round in Table IV. As shown in Table VI, harsh non-IIDness ($\alpha = 0.1$) with poor channel conditions makes FL convergence hard. SlimFL, on average, produces a 3.6x reduction in total computing costs and a 2.9x reduction in total communication costs until their convergences. In addition, Fig. 3 shows the energy efficiency results corresponding to accuracy. As shown in Fig. 3(a), SlimFL requires only the 32.63% of Vanilla FL-1.5x's total communication cost for model convergence. Similarly, the computing cost for SlimFL is the 44.37% of Vanilla FL-1.5x's as shown in Fig. 3(b). This increased energy efficiency is according to SlimFL's faster convergence even in non-IID and/or bad channel circumstances caused by SC and SD.

3) Robustness to Poor Channels: SlimFL and Vanilla FL both achieve high accuracy when channel conditions are good, as shown in Fig. 5 and Table VII. When the channel condition deteriorates from good to poor, however, as shown in Fig. 5(c,f) and Table VII, Vanilla FL1.0x's maximum accuracy at $\alpha = 10$ declines from 86 to 82 percent. SlimFL-1.0x, on the other hand, retains its maximum accuracy of 87 percent at $\alpha = 10$ in both good and bad channel conditions. Additionally, at $\alpha = 0.1$, SlimFL-1.0x outperforms Vanilla FL-1.0x by 18% in terms of top-1 accuracy, despite the fact that Vanilla FL-1.0x consumes more communication and computing resources. Additionally, as the channel condition degrades, the std of Vanilla FL-1.0x's top-1 accuracy increases by up to 59%, while SlimFL's std increases by just 31%. These findings show SlimFL's resistance to poor channels conditions, as well as its resistance to non-IID data distributions (low α) and communication efficiency.

4) Robustness to Non-IID Data: SlimFL-0.5x has a stable convergence under $\alpha = 0.1$ condition, as indicated in Fig. 5(d–f) and Table VII In poor channel conditions and with the non-IID distribution ($\alpha = 0.1$), Vanilla FL-0.5x and Vanilla FL-1.0x display the std of 8.3 and 9.2. On the other hand, SlimFL-0.5x and SlimFL-1.0x both demonstrate the std of 2.4 and 2.9 with top-1 accuracy. This propensity persists even when $\alpha = 1$, $\alpha = 10$ are prevalent. SlimFL-1.0x and SlimFL-0.5x have a smaller coefficient of variation than Vanilla FL-1.0x and Vanilla FL-0.5x. This accentuates SlimFL's robustness to non-IID data on poor channels.

5) Scalability: As shown in Fig. 6, SlimFL's accuracy increases as the number of federating devices increases. The SlimFL-0.5x achieves an accuracy of up to 79%, while the SlimFL-1.0x reaches an accuracy of up to 85%. Additionally, with non-IIDness ($\alpha = 0.1$) and 70 federating local devices, SlimFL-0.5x achieves a greater level of accuracy than SlimFL-1.0x with 20 federating local devices. Based on the experimental results, it is expected that when constructing a FL system using non-IID datasets, optimality can be obtained by adjusting the number of local devices and width through SlimFL adaption.

C. Ablation Studies on Optimization and Local Training

1) Effectiveness of SC Power Allocation Optimization: We present a method for optimizing our suggested model by modifying the ST parameter w_i. The numerical results of optimal λ^* are shown in Fig. 7. The derivative of the Tayler expansion is used in Proposition 1 to compute λ^*. The ideal power allocation factor is $\lambda^* = 0.662$, which is the same value as the numerical optimum, as determined by the analytical solution. The simulation was carried out using the baseline with $\lambda = 0.8$ in a non-IID settings (i.e., $\alpha = 0.1$) to verify the recommendation from Proposition 1. The experiment's result is shown in Fig. 8(a). The top-1 accuracy with λ^* is 6.4% higher than top-1 accuracy with 0.5x and 8.8% higher than top-1 accuracy with 1.0x. To put it another way, Proposition 1 acts as a guiding principle in SlimFL.

2) Effectiveness of ST Ratio Optimization: When $w_1 = \cdots = w_S = \frac{1}{S}$ in Proposition 2, SlimFL has a tight bound.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

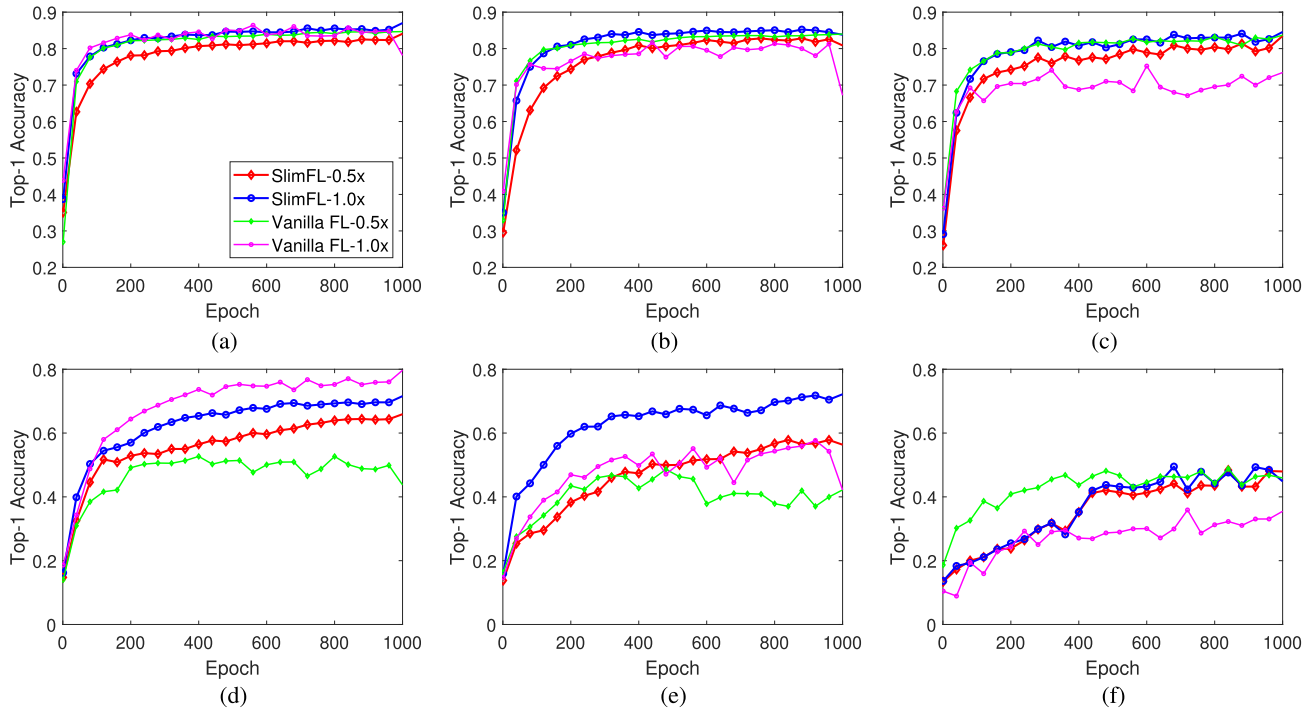


Fig. 5. Test accuracy in various channel noise conditions (on average). (a–c) are with $\alpha = 1$, (d–f) are with $\alpha = 0.1$.

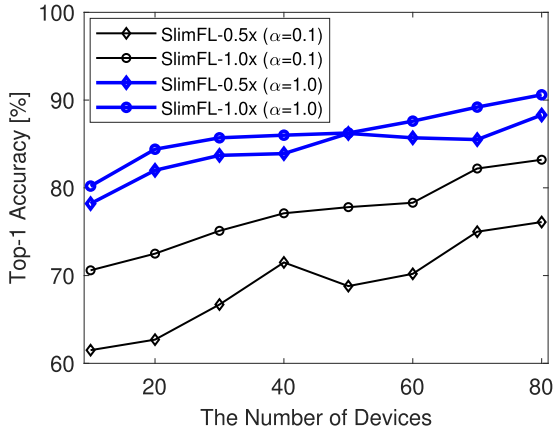


Fig. 6. Top-1 accuracy with the different number of devices.

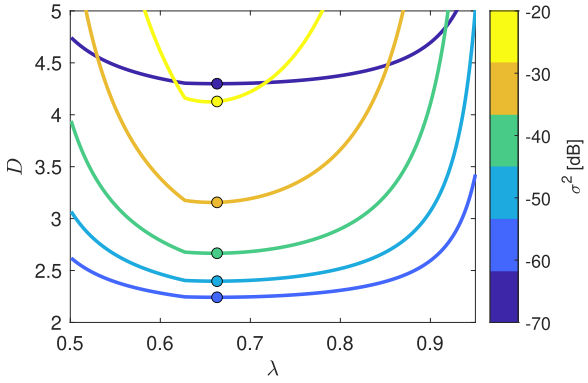


Fig. 7. SC power allocation ratio λ versus D $(= p_1^{-1} + p_2^{-1})$.

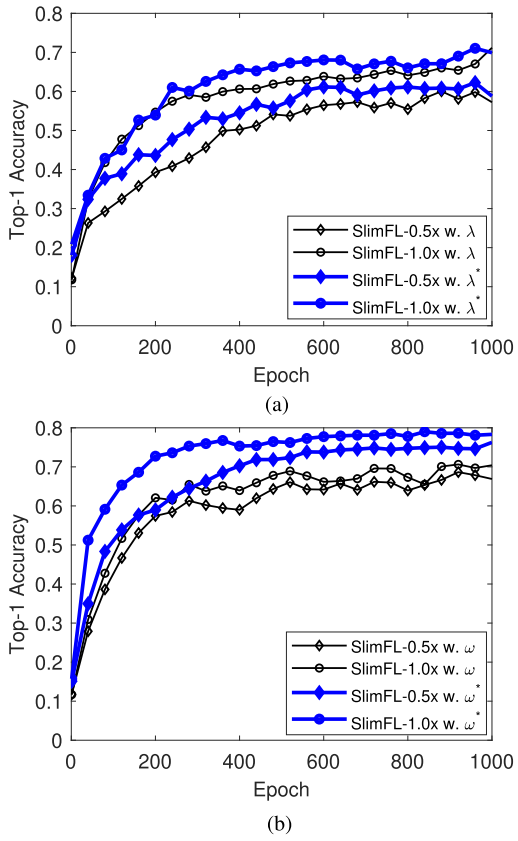


Fig. 8. Top-1 accuracy under optimal and non-optimal design parameters: (a) $\lambda^* = 0.663$ and $\lambda = 0.8$, and (b) $w_1^* = w_2^* = 0.5$ and ($w_1 = 0.3$, and $w_2 = 0.7$) with $\alpha = 0.1$.

Since $S = 2$ is taken into account in our proposed scheme, all hyperparameters that make up ST should be 0.5, i.e., $(w_1^*, w_2^*) = (0.5, 0.5)$. To verify Proposition 2, we construct a baseline with $(w_1, w_2) = (0.3, 0.7)$. Fig. 8(b) depicts the performance with different w_i. Under the optimal ST scheme, top-1 accuracy reaches 78 percent, whereas baseline accuracy is 69 percent. As a result, the ST guideline has a good impact on SlimFL's performance.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

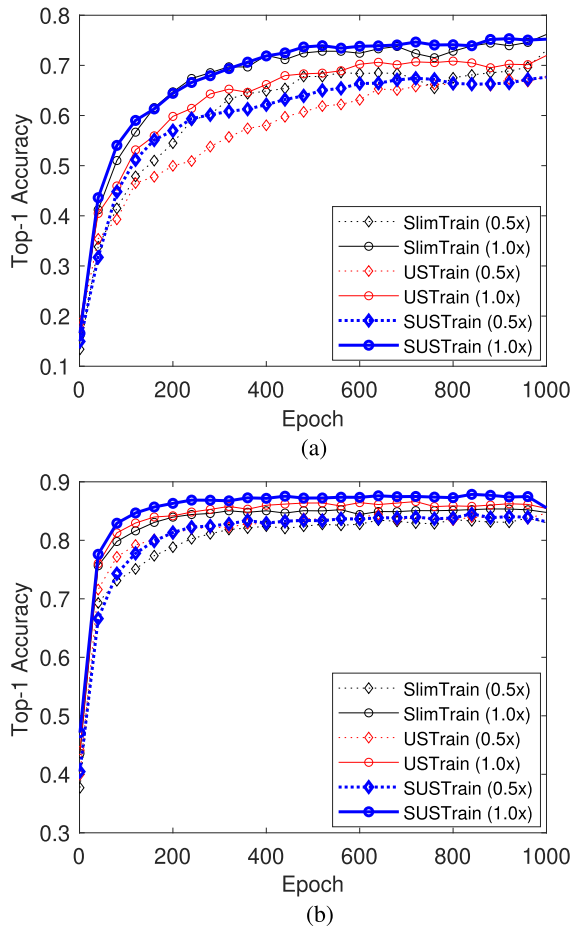


Fig. 9. Comparison of SNN training algorithms: SlimTrain [21], USTrain [20], and our proposed SUSTrain ($K = 10$, $\sigma^2 = -30$dB).

3) Effectiveness of the Proposed Local Training Algorithm: To configure out the superiority of the proposed local training algorithm (i.e., SUSTrain), we compare SUSTrain with SlimTrain [21] and USTrain [20] which are not only well-known but also state-of-the-art SNN training algorithms. The two training algorithms are presented in **Algorithm 1** and **Algorithm 2**). We investigate the performance between training algorithm in the scheme with ten local devices and $\sigma^2 = -30$dB in two data distributions (i.e., $\alpha = 0.1, 1.0$). As shown by experiments in Fig. 9(a), USTrain is unfit for SlimFL with high non-IID condition (i.e., $\alpha = 0.1$), where SlimTrain even outperforms USTrain. With IID data distribution (i.e., $\alpha = 1.0$), USTrain outperforms SlimTrain. Fig. 9 corroborates that regardless of the data distributions, SUSTrain achieves high accuracy with fast convergence, as opposed to USTrain which is effective only under IID data distributions (i.e., $\alpha = 1.0$). We conjecture that the problem comes from the use of outdated teacher's knowledge in USTrain. In USTrain, the teacher's logit is set as the value before updating the teacher's model and is compared with a student after updating the teacher's model. Non-IID data distributions exacerbate this mismatch, where the full-width teacher model is significantly updated in the first epoch after downloading the global model due to the huge gap between local and global models.

TABLE VIII
PERFORMANCE OF VARIOUS WIDTH CONFIGURATIONS

Metric	SNN 1/6x	2/6x	3/6x	4/6x	5/6x	6/6x
Top-1 Accuracy (%)	72.6	77.3	82.5	84.6	84.6	85.1
Computation Cost per Image (MFLOPS)	0.23	0.45	0.97	1.73	2.71	3.82

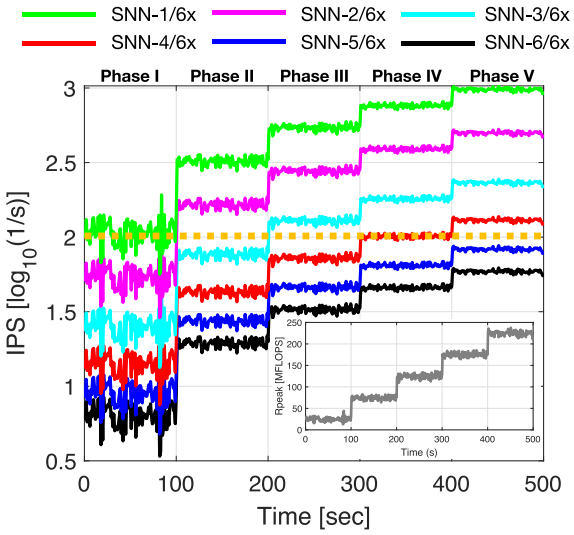


Fig. 10. Image processing per second of various width configurations given time-varing theoretical CPU resources (i.e., $\mathsf{R}_{\mathsf{peak}}$).

D. Impact on Heterogeneous Energy Capacity

To verify the impact on heterogeneous computing capacities of devices, we investigate six-width configurations (e.g., 1/6x, 2/6x, $\cdots$, and 6/6x). First, we train SNNs with the default settings of SlimFL ($\alpha = 10$, $K = 10$, $\sigma^2 = -30$dB). Then, we measure top-1 accuracy and calculate FLOPS. Its specification is presented in Table VIII. Second, we investigate the dynamic width adaptation to the device, which has a time-varying theoretical peak performance of the CPU, denoted as $\mathsf{R}_{\mathsf{peak}}$ (MFLOPS). We set $\mathsf{R}_{\mathsf{peak}} \in [20, 230]$ (MFLOPS). Lastly, we calculate the number of image processing per second (IPS) by the following metric:

$$\mathsf{IPS} = \frac{\mathsf{R}_{\mathsf{peak}}\ [\mathsf{MFLOPS/sec}]}{\mathsf{Computational\ Cost\ per\ Image\ [MFLOPS]}}. \quad (26)$$

Fig. 10 shows IPS of multi-width configurations in log-scale, and $\mathsf{R}_{\mathsf{peak}}$ over time. As shown in Fig. 10, five different $\mathsf{R}_{\mathsf{peak}}$ phases exist. Suppose that the mobile devices require 100 images per second, i.e., $\mathsf{IPS}_{target} = 100$ marked as the *yellow dashed line*. In phase I in Fig. 10, the IPS requirement is only satisfied when using SNN-1/6x. In phase II, the IPS requirement is satisfied when using SNN-1/6x and SNN-2/6x. However, it is recommended to use SNN-2/6x, because the top-1 accuracy increases 4.7%. Similarly, SNN-3/6x, SNN-4/6x are recommended to be used in phase III, and phase IV, and V, respectively. Furthermore, we can use six model configurations with the memory usage of SNN-1.0x. If vanilla NN is used, 3.5x more memory is required than using SNN. Thus,

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

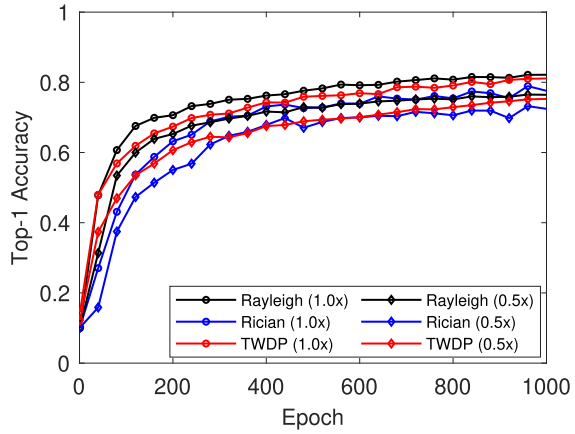


Fig. 11. Top-1 accuracy with the different fading effects.

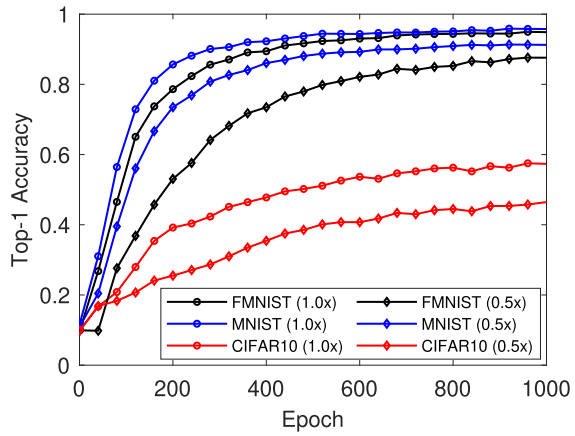
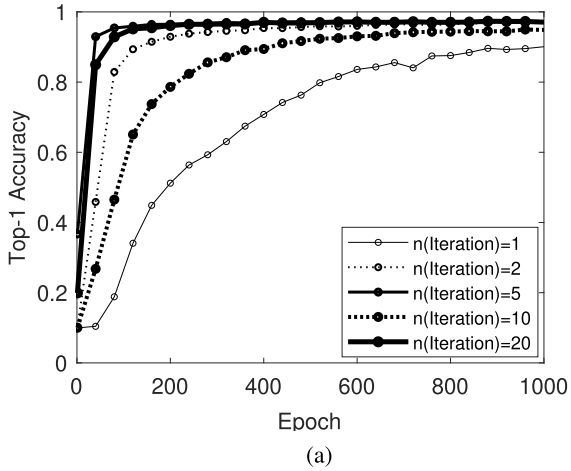


Fig. 12. Top-1 accuracy with the different datasets.



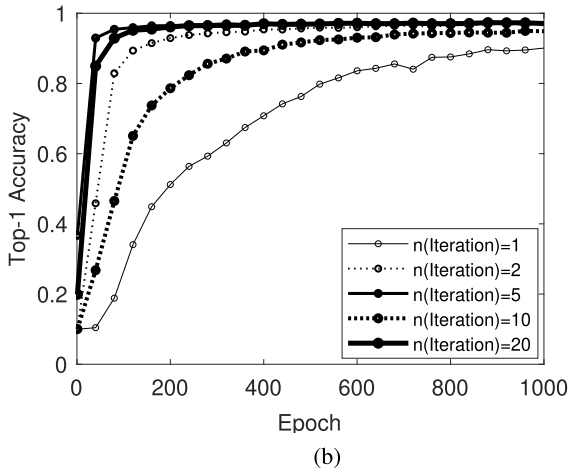


Fig. 13. Top-1 accuracy with different number of local iterations.

using SNN is an adjustable solution for computational energy heterogeneous conditions.

E. Feasibility Studies on Channel and Training Environments

We investigate the generalization of SlimFL across various datasets and the different numbers of local iterations in various channel models.

1) Impact of Channel Models: We design additional simulations to verify SlimFL works in more realistic channel models (e.g., Rician fading [30] and Two-wave with diffuse power fading (TWDP) [41]). In indoor wireless communication, Rician fading is superior than Rayleigh fading. In addition, TWDP is used in air, space or maritime wireless communication, where the channel fading is worse than Rayleigh [42]. We simulate Rician fading with K-factor 3.5, where the fading follows Rician distributions $\chi \sim \text{Rice}(0.5577, 0.2106)$. (K, Δ)-factor of TWDP fading is set to $(3.5, 0.1)$. All communication settings without fading effect are identical to Table II. Fig. 11 is the result of the simulation. As shown in Fig. 11, the top-1 accuracy is found to be high in the order of Rayleigh fading, TWDP fading, and Rician fading channel models. SlimFL-1.0x always outperforms SlimFL-0.5x. Thus, SlimFL performs well in a variety of channel models.

2) Impact of Datasets: We conduct additional simulations to verify whether SlimFL operates with other datasets (e.g., CIFAR10 [43] and MNIST [44]). CIFAR10 dataset consists of 60,000 color images with size 32×32. MNIST is a handwritten digit classification dataset. It consists of 60,000 grayscale images with size 28×28. Both image datasets have 10 classes, i.e., digits. CIFAR10 is known to be a more challenging problem, which is harder than MNIST or FMNIST. The simulation is conducted with $\alpha = 1.0, K = 10$, and $\sigma^2 = -30\text{dB}$. Fig. 12 is the result of the simulation. SlimFL with MNIST dataset shows the best top-1 accuracy, FMNIST data set shows the next highest accuracy, and CIFAR10 shows the lowest accuracy. In terms of the performance between 1.0x and 0.5x, SlimFL-1.0x is always higher than SlimFL-0.5x. In other words, we confirm that our proposed SlimFL works well regardless of the dataset.

3) Impact of Local Training Iterations: This paper adopts local iterations per communication round as 1 for mathematical tractability. Thus, the convergence bound does not contain the local iteration. According to [33], the over-small or over-large local iterations cause high communication and computational cost, while the optimal local iteration exists. We conduct additional simulations to verify the SlimFL operation under different numbers of local iterations (i.e., 1, 2, 5, 10, and

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

20). As mentioned above, FedAvg does not guarantee that many local iterations are proportional to the performance [32]. Similarly in our simulation as shown in Fig. 13, the top-1 accuracy of both SlimFL-1.0x and SlimFL-0.5x is high and is on the order of $\{5, 20, 2, 10, 1\}$. The empirical result shows that there exist at least two local extremums. One is located in $[2, 10]$, and the other is in $[5, \infty]$, respectively. In other words, there are at least two local optima corresponding to local iterations in $[1, \infty]$. Note that these local optima do not indicate global optima.

VII. Concluding Remarks

Existing FL solutions are incapable of adjusting flexibly to devices with varying amounts of available energy and channel throughput without jeopardizing communication or energy efficiency. To overcome this problem, we propose SlimFL, a novel framework for FL over SNNs that utilizes superpositioned training for local SNN training and superposition coding for trained model aggregation. Extensive analyses and simulations show that SlimFL is an energy- and communication-efficient solution in a variety of communication conditions and data distributions. SlimFL achieves higher accuracy and faster convergence while using less energy than vanilla FL, which requires 2X more communication resources. Additionally, studying the impact of more adjustable SNN width levels could be an interesting topic for future work. Another interesting direction is to apply SlimFL for multitask learning with various width configurations in different tasks.

Appendix I
Proof of Theorem 1

A. Proof of Lemma 1

Proof: According to f_t in (13) and Assumption 3,

$$\|f_t - \bar{f}_t\|^2 = \left\| \frac{1}{Kp_1} \sum_{k\in\mathsf{H}\cup\mathsf{F}} (g_t^k - \bar{g}_t^k) \odot \Xi + \frac{1}{Kp_2} \sum_{k\in\mathsf{F}} (g_t^k - \bar{g}_t^k) \odot \Xi^{-1} \right\|^2 \tag{27}$$

$$\leq \frac{2}{Kp_1} \sum_{k\in\mathsf{H}} \|(g_t^k - \bar{g}_t^k) \odot \Xi\|^2 + \frac{2}{Kp_2} \sum_{k\in\mathsf{F}} \|(g_t^k - \bar{g}_t^k) \odot \Xi^{-1}\|^2 \tag{28}$$

$$\leq \frac{2}{Kp_1} \sum_{k\in\mathsf{H}} \|g_t^k - \bar{g}_t^k\|^2 + \frac{2}{Kp_2} \sum_{k\in\mathsf{F}} \|g_t^k - \bar{g}_t^k\|^2, \tag{29}$$

where the first inequality follows from the Cauchy–Schwarz (C-S) inequality, and the last step is because $\|X \odot \Xi\|^2 \leq \|X\|^2$. Similarly, we have

$$\left\|g_t^k - \bar{g}_t^k\right\|^2 \left\| \sum_{i=1}^{2} w_i (\nabla F^k(\theta_t, \zeta_t^k) - \nabla F^k(\theta_t)) \odot \Xi_i \right\|^2 \leq 2 \sum_{i=1}^{2} w_i^2 \left\| \nabla F^k(\theta_t, \zeta_t^k) - \nabla F^k(\theta_t) \right\|^2. \tag{30}$$

Taking an expectation at both sides yields

$$\mathbb{E}\|g_t^k - \bar{g}_t^k\|^2 \leq 2\sigma_k^2 \sum_{i=1}^{2} w_i^2. \tag{31}$$

Combining (29) and (31) finalizes the proof. □

B. Proof of Lemma 2

Proof: According to (13), we have

$$\|\theta_{t+1} - \theta^*\|^2 = \|\theta_t - \eta_t f_t - \theta^* - \eta_t \bar{f}_t + \eta_t \bar{f}_t\|^2 \tag{32}$$

$$= \underbrace{\|\theta_t - \theta^* - \eta_t \bar{f}_t\|^2}_{A_1} + \underbrace{2\eta_t \langle \theta_t - \theta^* - \eta_t f_t, \bar{f}_t - f_t \rangle}_{A_2} + \underbrace{\eta_t^2 \|f_t - \bar{f}_t\|^2}_{A_3} \tag{33}$$

$$= \|\theta_t - \theta_t^*\|^2 - \underbrace{2\eta_t \langle \theta_t - \theta^*, \bar{f}_t \rangle}_{B_1} + \underbrace{\eta_t^2 \|\bar{f}_t\|^2}_{B_2} + A_2 + A_3. \tag{34}$$

Here, $\mathbb{E}[A_2] = 0$ due to $\mathbb{E}(f_t) = \bar{f}_t$, and A_3 is bounded according to Lemma 1. Note that $\bar{f}_t = \mathbb{E}[f_t] = \mathbb{E}[\nabla F(\theta_t)] = \nabla \mathbb{E}[F(\theta_t)]$, and $\mathbb{E}[F]$ inherits the μ-strong convexity and L-smoothness from F. By the L-smoothness of $\mathbb{E}[F]$, we have

$$\|\bar{f}_t\|^2 \leq 2L(\mathbb{E}[F(\theta_t) - F(\theta^*)]), \tag{35}$$

showing the boundness of B_2. Next, by the μ-strong convexity of $\mathbb{E}[F]$, we have

$$\langle \theta^* - \theta_t, \bar{f}_t \rangle \leq \mathbb{E}[F(\theta^*) - F(\theta_t)] - \frac{\mu}{2} \|\theta_t - \theta^*\|^2, \tag{36}$$

proving the boundness of B_1. Applying the bounds of B_1 and B_2, we obtain

$$A_1 \leq (1 - \mu\eta_t) \|\theta_t - \theta^*\|^2 - 2\eta_t (1 - L\eta_t) \mathbb{E}[F(\theta_t) - F(\theta^*)], \tag{37}$$

where the last term on the RHS vanishes for $\eta_t < \frac{1}{L}$. Taking the expectation at both sides completes the proof. □

C. Completing the Proof of Theorem 1

Proof: Since $\eta_t = \frac{2}{\mu t + 2L - \mu} \leq \frac{1}{L}$, applying Lemma 2, we have

$$\Delta_{t+1} \leq (1 - \mu\eta_t) \Delta_t + \eta_t^2 B. \tag{38}$$

By induction, we aim to show that $\Delta_t \leq \frac{v}{t + 2\kappa - 1}$ where $\kappa = \frac{L}{\mu}$ and $v = \max\{2\kappa\Delta_1, 4B/\mu^2\}$ as elaborated next. By the definition of v, it is trivial that $\Delta_1 \leq \frac{v}{2\kappa}$. Assuming that $\Delta_{t'} \leq \frac{v}{t' + 2\kappa - 1}$ holds, we have

$$\Delta_{t'+1} \leq (1 - \mu\eta_{t'}) \Delta_{t'} + \eta_{t'}^2 B \tag{39}$$

$$\leq \left(1 - \frac{2}{t' + 2\kappa - 1}\right) \frac{v}{t' + 2\kappa - 1} + \frac{4B/\mu^2}{(t' + 2\kappa - 1)^2} \tag{40}$$

$$= \frac{(t' + 2\kappa - 2)v - (v - 4B/\mu^2)}{(t' + 2\kappa - 1)^2} \tag{41}$$

$$\leq \frac{t' + 2\kappa - 2}{(t' + 2\kappa - 1)^2} v \leq \frac{v}{t' + 2\kappa}, \tag{42}$$

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

which proves that $\Delta_t \leq \frac{v}{t+2\kappa-1}$. For $t=1$, we obtain

$$v = \max\{2\kappa\Delta_1, \frac{4B}{\mu^2}\} \leq 2\kappa\Delta_1 + \frac{4B}{\mu^2}. \quad (43)$$

Finally, by the L-Smoothness of F, one has

$$\mathbb{E}[F(\theta_t)] - F^* = \mathbb{E}[F(\theta_t) - F(\theta^*)] \leq \frac{L}{2}\mathbb{E}\|\theta_t - \theta^*\|^2. \quad (44)$$

Applying Lemma 2 with the results above, we have

$$\mathbb{E}\|\theta_t - \theta^*\|^2 \leq \frac{v}{t+2\kappa-1} \leq \frac{2}{\mu} \cdot \frac{\mu L\Delta_1 + 2B}{\mu t + 2L - \mu}, \quad (45)$$

which completes the proof of the theorem. □

REFERENCES

[1] H. Baek et al., "Joint superposition coding and training for federated learning over multi-width neural networks," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2022, pp. 1729–1738.

[2] S. Savazzi, M. Nicoli, M. Bennis, S. Kianoush, and L. Barbieri, "Opportunities of federated learning in connected, cooperative, and automated industrial systems," *IEEE Commun. Mag.*, vol. 59, no. 2, pp. 16–21, Feb. 2021.

[3] N.-N. Dao et al., "Hit ratio and content quality tradeoff for adaptive bitrate streaming in edge caching systems," *IEEE Syst. J.*, vol. 15, no. 4, pp. 5094–5097, Dec. 2021.

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, Fort Lauderdale, FL, USA, Apr. 2017, pp. 1273–1282.

[5] N. H. Tran, W. Bao, A. Zomaya, N. M. Nh, and C. S. Hong, "Federated learning over wireless networks: Optimization model design and analysis," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Paris, France, Apr. 2019, Apr. 2019, pp. 1387–1395.

[6] C. T. Dinh et al., "Federated learning over wireless networks: Convergence analysis and resource allocation," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 398–409, Feb. 2021.

[7] D. Kwon, J. Jeon, S. Park, J. Kim, and S. Cho, "Multiagent DDPG-based deep learning for smart ocean federated learning IoT networks," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9895–9903, Oct. 2020.

[8] T. Cover, "Broadcast channels," *IEEE Trans. Inf. Theory*, vol. IT-18, no. 1, pp. 2–14, Jan. 1972.

[9] B. McMahan and D. Ramage, "Federated learning: Collaborative machine learning without centralized training data," *Google AI Blog*, Apr. 2017.

[10] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2021, pp. 1–27.

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 429–450.

[12] S. J. Reddi et al., "Adaptive federated optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–38.

[13] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1935–1949, Mar. 2020.

[14] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 269–283, Oct. 2020.

[15] Y. Koda et al., "Communication-efficient multimodal split learning for mmWave received power prediction," *IEEE Commun. Lett.*, vol. 24, no. 6, pp. 1284–1288, Jun. 2020.

[16] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, San Juan, Puerto Rico, May 2016, pp. 1–14.

[17] G. E. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Montreal, QC, Canada, Dec. 2015, pp. 1–9.

[18] D. Kim, J. Kim, J. Kwon, and T.-H. Kim, "Depth-controllable very deep super-resolution network," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Budapest, Hungary, Jul. 2019, pp. 1–8.

[19] H. Hu, D. Dey, M. Hebert, and J. A. Bagnell, "Learning anytime predictions in neural networks via adaptive loss balancing," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Honolulu, HI, USA, Jan. 2019, pp. 3812–3821.

[20] J. Yu and T. S. Huang, "Universally slimmable networks and improved training techniques," in *Proc. CVPR*, Oct. 2019, pp. 1803–1811.

[21] J. Yu, L. Yang, N. Xu, J. Yang, and T. Huang, "Slimmable neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, USA, May 2019, pp. 1–12.

[22] C. Li, G. Wang, B. Wang, X. Liang, Z. Li, and X. Chang, "Dynamic slimmable network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 8607–8617.

[23] K. Higuchi and A. Benjebbour, "Non-orthogonal multiple access (NOMA) with successive interference cancellation for future radio access," *IEICE Trans. Commun.*, vol. 98, no. 3, pp. 403–414, 2015.

[24] J. Choi, "Minimum power multicast beamforming with superposition coding for multiresolution broadcast and application to NOMA systems," *IEEE Trans. Commun.*, vol. 63, no. 3, pp. 791–800, Mar. 2015.

[25] J. Choi, "On the power allocation for a practical multiuser superposition scheme in NOMA systems," *IEEE Commun. Lett.*, vol. 20, no. 3, pp. 438–441, Mar. 2016.

[26] Z. Ding et al., "Application of non-orthogonal multiple access in LTE and 5G networks," *IEEE Commun. Mag.*, vol. 55, no. 2, pp. 185–191, Feb. 2017.

[27] V. D. Tuong, T. P. Truong, T.-V. Nguyena, W. Noh, and S. Cho, "Partial computation offloading in NOMA-assisted mobile-edge computing systems using deep reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 17, pp. 13196–13208, Sep. 2021.

[28] D. N. C. Tse and P. Viswanath, *Fundamentals of Wireless Communications*. Cambridge, U.K.: Cambridge Univ. Press, 2005.

[29] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," 2021, *arXiv:2103.13630*.

[30] A. F. Molisch, *Wireless Communications*, 2nd ed. Hoboken, NJ, USA: Wiley, 2011.

[31] J. Choi, "Joint rate and power allocation for NOMA with statistical CSI," *IEEE Trans. Commun.*, vol. 65, no. 10, pp. 4519–4528, Oct. 2017.

[32] H. Wang, M. Yurochkin, Y. Sun, D. S. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2020, pp. 1–16.

[33] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020, pp. 1–26.

[34] A. Khaled, K. Mishchenko, and P. Richtárik, "Tighter theory for local SGD on identical and heterogeneous data," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, Palermo, Sicily, Italy, Aug. 2020, pp. 4519–4529.

[35] S. U. Stich, "Local SGD converges fast and communicates little," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Vancouver, BC, Canada, Apr./May 2018, pp. 1–19.

[36] A. Mohtashami, M. Jaggi, and S. U. Stich, "Simultaneous training of partially masked neural networks," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, Mar. 2022, pp. 1–18.

[37] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.

[38] T.-M. Harry Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," in *Proc. Conf. Neural Inf. Process. Syst. (NIPS) Federated Learn. Data Privacy Confidentiality Workshop*, Sep. 2019, pp. 1–5.

[39] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[40] D. Hernandez and T. B. Brown, "Measuring the algorithmic efficiency of neural networks," May 2020, *arXiv:2005.04305*.

[41] G. D. Durgin, T. S. Rappaport, and D. A. de Wolf, "New analytical models and probability density functions for fading in wireless communications," *IEEE Trans. Commun.*, vol. 50, no. 6, pp. 1005–1015, Jun. 2002.

[42] C. X. Wang, J. Huang, H. Wang, X. Gao, X. You, and Y. Hao, "6G wireless channel measurements and models: Trends and challenges," *IEEE Veh. Technol. Mag.*, vol. 15, no. 4, pp. 22–32, Dec. 2020.

[43] A. Krizhevsky, V. Nair, and G. Hinton. *Cifar-10 (Canadian Institute for Advanced Research)*. Accessed: Dec. 2009. [Online]. Available: http://www.cs.toronto.edu/~kriz/cifar.html

[44] L. Deng, "The MNIST database of handwritten digit images for machine learning research," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, Nov. 2012.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.

Won Joon Yun (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering from Korea University, Seoul, South Korea, where he is currently pursuing the Ph.D. degree in electrical and computer engineering. He was a Visiting Researcher at Cipherome Inc., San Jose, CA, USA, in Summer 2022, and a Visiting Researcher at the University of Southern California, Los Angeles, CA, USA, in Winter 2022, for a joint project with Prof. Andreas F. Molisch at the Ming Hsieh Department of Electrical and Computer Engineering, USC Viterbi School of Engineering.

Yunseok Kwak received the B.S. degree in mathematics from Yonsei University, Seoul, South Korea, in 2021. He is currently pursuing the Ph.D. degree in electrical and computer engineering with Korea University, Seoul.

Hankyul Baek (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering from Korea University, Seoul, South Korea, in 2020, where he is currently pursuing the Ph.D. degree in electrical and computer engineering. He was at LG Electronics, Seoul, from 2020 to 2021. His current research interests include multi-agent deep reinforcement learning and its augmented reality applications.

Soyi Jung (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical and computer engineering from Ajou University, Suwon, South Korea, in 2013, 2015, and 2021, respectively.

She was a Researcher at the Korea Testing and Research (KTR) Institute, Gwacheon, South Korea, from 2015 to 2016, an Assistant Professor at Hallym University, Chuncheon, South Korea, from 2021 to 2022, a Visiting Scholar at the Donald Bren School of Information and Computer Sciences, University of California, Irvine, CA, USA, from 2021 to 2022, and a Research Professor at Korea University, Seoul, South Korea, in 2021. She has been an Assistant Professor with the Department of Electrical of Computer Engineering, Ajou University, since September 2022. She was a recipient of the Best Paper Award from KICS in 2015, the Young Women Researcher Award from WISET and KICS in 2015, the Bronze Paper Award from IEEE Seoul Section Student Paper Contest in 2018, the ICT Paper Contest Award by Electronic Times in 2019, and the IEEE International Conference on Information Networking (ICOIN) Best Paper Award in 2021.

Mingyue Ji (Member, IEEE) received the B.E. degree in communication engineering from the Beijing University of Posts and Telecommunications, China, in 2006, the M.Sc. degree in electrical engineering from the Royal Institute of Technology, Sweden, in 2008, the M.Sc. degree in electrical engineering from the University of California, Santa Cruz, in 2010, and the Ph.D. degree from the Ming Hsieh Department of Electrical Engineering, University of Southern California, in 2015.

From 2015 to 2016, he was a Staff II System Design Scientist at Broadcom Corporation. He is an Associate Professor with the Electrical and Computer Engineering Department, The University of Utah. He received the IEEE Communications Society Leonard G. Abraham Prize for the Best IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS Paper in 2019, the Best Paper Award from IEEE GLOBECOM 2021 Conference, the Best Paper Award from IEEE ICC 2015 Conference, the Best Student Paper Award from IEEE European Wireless 2010 Conference, and the USC Annenberg Fellowship from 2010 to 2014. Since 2020, he has been an Associate Editor of IEEE TRANSACTIONS ON COMMUNICATIONS.

Mehdi Bennis (Fellow, IEEE) is a Tenured Full Professor with the Centre for Wireless Communications, University of Oulu, Finland, an Academy of Finland Research Fellow, and the Head of the Intelligent Connectivity and Networks/Systems Group (ICON). He has published more than 200 research papers in international conferences, journals, and book chapters. His main research interests include radio resource management, heterogeneous networks, game theory, and distributed machine learning in 5G networks and beyond.

He was a recipient of several prestigious awards, including the 2015 Fred W. Ellersick Prize from the IEEE Communications Society, the 2016 Best Tutorial Prize from the IEEE Communications Society, the 2017 EURASIP Best Paper Award for the *EURASIP Journal on Wireless Communications and Networking*, the All-University of Oulu Award for research, the 2019 IEEE ComSoc Radio Communications Committee Early Achievement Award, and the 2020 Clarivate Highly Cited Researcher by the Web of Science. He is the Editor of IEEE TRANSACTIONS ON COMMUNICATIONS and the Specialty Chief Editor of *Data Science for Communications* (Frontiers) and the *Frontiers in Communications and Networks*.

Jihong Park (Senior Member, IEEE) received the B.S. and Ph.D. degrees from Yonsei University, South Korea. He is currently a Lecturer (an Assistant Professor) with the School of Information Theory, Deakin University, Australia. His research interests include ultra-dense/ultra-reliable/mmWave system designs and distributed learning/control/ledger technologies and their applications for beyond-5G/6G communication systems. He served as a Conference/Workshop Program Committee Member for IEEE GLOBECOM, ICC, and WCNC, and for NeurIPS, ICML, and IJCAI. He is an Associate Editor of *Data Science for Communications* (Frontiers) and a Review Editor of *Aerial and Space Networks* (Frontiers).

Joongheon Kim (Senior Member, IEEE) received the B.S. and M.S. degrees in computer science and engineering from Korea University, Seoul, South Korea, in 2004 and 2006, respectively, and the Ph.D. degree in computer science from the University of Southern California (USC), Los Angeles, CA, USA, in 2014.

He was at LG Electronics, Seoul, from 2006 to 2009, InterDigital, San Diego, CA, USA, in 2012, Intel Corporation, Santa Clara, Silicon Valley, CA, USA, from 2013 to 2016, and Chung-Ang University, Seoul, from 2016 to 2019. Since 2019, he has been with Korea University, where he is currently an Associate Professor.

Dr. Kim is a Distinguished Lecturer of IEEE Communications Society (ComSoc) from 2022 to 2023 and IEEE Systems Council from 2022 to 2024. He was a recipient of the Annenberg Graduate Fellowship for his Ph.D. admission from USC in 2009, the Intel Corporation Next Generation and Standards (NGS) Division Recognition Award in 2015, the IEEE SYSTEMS JOURNAL Best Paper Award in 2020, the IEEE ComSoc Multimedia Communications Technical Committee (MMTC) Outstanding Young Researcher Award in 2020, the IEEE ComSoc MMTC Best Journal Paper Award in 2021, and the Best Special Issue Guest Editor Award by *ICT Express* (Elsevier) in 2022. He also received numerous awards from IEEE international conferences, including the IEEE International Conference on Information Networking (ICOIN) Best Paper Award in 2021, the IEEE Vehicular Technology Society (VTS) Seoul Chapter Awards for APWCS in 2019 and 2021, and the IEEE ICTC Best Paper Award in 2022. He is the Editor of IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE TRANSACTIONS ON MACHINE LEARNING IN COMMUNICATIONS AND NETWORKING, *IEEE Communications Standards Magazine*, *Computer Networks* (Elsevier), and *ICT Express* (Elsevier).

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on September 27,2026 at 14:42:43 UTC from IEEE Xplore. Restrictions apply.